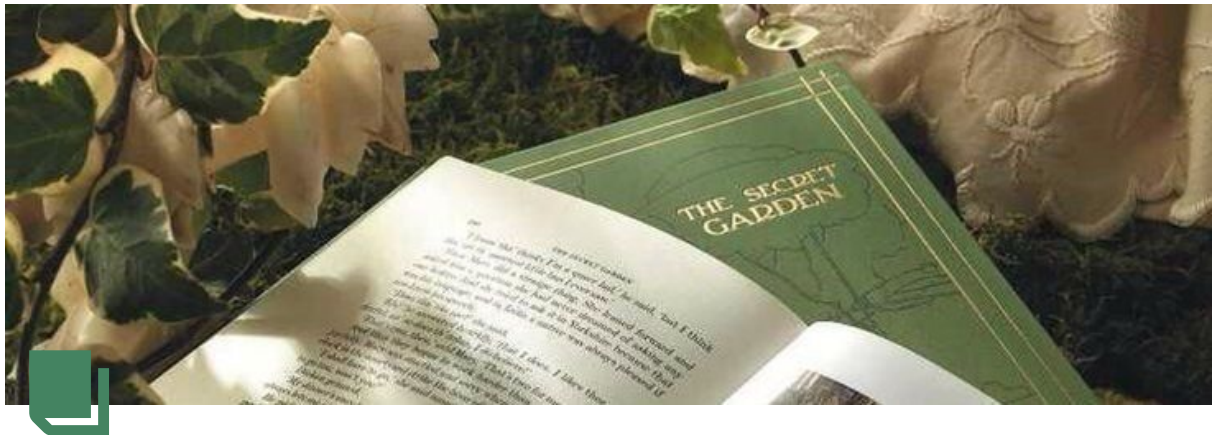




Introdução e fundamentos - Aula 1

🕒 Criado em	7 de setembro de 2026 11:16
🏷 Tags	

A primeira coisa que precisamos entender é que a Engenharia de Programas não começa perguntando:

| "Como escrever o código?"

Ela começa com uma preocupação maior:

| **Como aplicar princípios de engenharia ao desenvolvimento e à execução de programas de computador?**

Isso muda bastante a perspectiva.

Um programador pode conseguir fazer um programa funcionar. Mas a Engenharia de Programas está interessada também em questões como:

- É **viável** desenvolver esse programa?
- Ele é **racional** considerando seus impactos?
- Ele é **seguro**?
- Ele utiliza os recursos de forma **eficiente**?
- Ele pode ser **mantido e modificado** adequadamente?

- Quanto custa sua execução?
- Como esse custo cresce quando o problema aumenta?

É daí que surgem os **cinco princípios básicos da engenharia** apresentados pelo professor.

1. Os cinco princípios básicos da Engenharia

1.1 Viabilidade

Antes de construir qualquer produto ou sistema, precisamos perguntar:

É realmente possível e justificável construí-lo?

Na Engenharia de Programas, a viabilidade possui três perspectivas principais.

Viabilidade técnica

Pergunta:

Temos tecnologia e recursos humanos/materiais para desenvolver o sistema?

Por exemplo, uma empresa quer desenvolver um sistema de reconhecimento de imagens.

Precisamos verificar:

- temos computadores capazes de executar o processamento?
- temos conhecimento técnico?
- temos programadores suficientes?
- as tecnologias necessárias existem?
- conseguimos contratar pessoas com as competências necessárias?

Se a tecnologia necessária não estiver disponível ou não houver recursos para desenvolvê-la, o projeto pode ser tecnicamente inviável.

Viabilidade econômica

Agora a pergunta muda:

■ Temos dinheiro suficiente para desenvolver e manter o projeto?

Não é apenas o custo de escrever o código.

Podemos ter:

- salários dos desenvolvedores;
- servidores;
- computadores;
- serviços de nuvem;
- licenças;
- banco de dados;
- internet;
- infraestrutura;
- testes;
- manutenção;
- suporte;
- marketing;
- custos operacionais.

Imagine uma empresa que estima gastar R\$ 500 mil para desenvolver um sistema, mas espera recuperar esse investimento somente depois de muitos anos.

Mesmo sendo tecnicamente possível, talvez **não seja economicamente interessante**.

Viabilidade comercial

Aqui a pergunta é:

■ Existe mercado para esse produto?

Não adianta conseguir construir um sistema excelente se ninguém quiser utilizá-lo ou pagar por ele.

Devemos considerar:

- existe demanda?
- quem são os usuários?
- qual preço o mercado aceita?
- existe concorrência?
- o produto consegue gerar retorno?
- existe uma estratégia de venda e distribuição?

Então podemos guardar:

VIABILIDADE

- Técnica → conseguimos construir?
- Econômica → conseguimos financiar?
- Comercial → existe mercado/retorno?

Essa distinção é **bem importante para prova**.

2. Racionalidade

Aqui começa uma parte que pode parecer abstrata inicialmente.

A ideia é que **não basta uma tecnologia ser possível**.

Precisamos analisar se seus benefícios justificam seus custos e riscos.

Em outras palavras:

Qual é a relação entre aquilo que estamos ganhando e os impactos que estamos produzindo?

Essa análise pode envolver aspectos:

- sociais;
- éticos;
- ambientais;
- ecológicos;
- econômicos.

Exemplo de mercado

Imagine um sistema de inteligência artificial utilizado para tomar decisões importantes sobre pessoas.

Tecnicamente, podemos conseguir desenvolver o sistema.

Mas isso não significa automaticamente que devemos colocá-lo em produção.

Precisamos questionar:

- ele pode gerar decisões injustas?
- os dados utilizados são adequados?
- existe risco de discriminação?
- quem será responsabilizado por uma decisão errada?
- quais são os benefícios?
- quais são os possíveis impactos negativos?

Esse é o papel da **racionalidade**.

3. Segurança

A segurança está relacionada à redução dos riscos associados ao sistema.

O professor apresentou uma relação conceitual:

| **Risco = Dano × Taxa provável de falha/acidente**

A ideia é bastante importante.

Não basta perguntar:

| "O sistema pode falhar?"

Quase qualquer sistema pode.

Precisamos perguntar:

| **"Se ele falhar, qual será o dano e qual é a probabilidade de isso acontecer?"**

Imagine dois sistemas.

Sistema A

Uma falha faz uma tela ficar desalinhada.

Dano: pequeno.

Sistema B

Uma falha em um sistema de controle pode provocar um acidente.

Dano: potencialmente muito maior.

Portanto, não podemos tratar os dois riscos da mesma maneira.

Tipos de danos

O professor apresentou quatro categorias:

- **material;**
- **ecológico;**
- **humano;**
- **financeiro.**

E existe uma observação importante:

Um dano que inicialmente não é financeiro pode acabar gerando consequências financeiras.

Por exemplo, um problema ambiental pode gerar:

```
Dano ambiental
  ↓
Multas
  ↓
Custos de recuperação
  ↓
Perda de produção
  ↓
Prejuízo financeiro
```

4. ALARA

Aqui temos um termo que vale muito a pena guardar:

ALARA — As Low As Reasonably Achievable

A ideia geral é:

| **Manter o risco tão baixo quanto razoavelmente possível.**

Não significa necessariamente que seja possível eliminar absolutamente todos os riscos.

Em engenharia, muitas vezes precisamos trabalhar com sistemas que possuem algum nível de risco.

Então buscamos reduzir esse risco para um nível considerado aceitável dentro das condições existentes.

5. Segurança de software

E aqui aparece uma conexão muito importante com programação.

Às vezes existe a ideia equivocada de que:

| "Software não machuca ninguém porque software é apenas código."

Mas software controla sistemas do mundo real.

Por exemplo:

```
Software
  ↓
Sistema de controle
  ↓
Equipamento físico
  ↓
Consequência no mundo real
```

Isso pode acontecer em:

- transporte;
- aviação;
- indústria;
- equipamentos médicos;
- sistemas financeiros;
- infraestrutura;

- redes elétricas.

Além disso, segurança também envolve **proteção de dados e controle de acesso**.

Um software pode causar problemas por:

1. Mau funcionamento

Um bug pode:

- corromper dados;
- gerar informações incorretas;
- interromper um serviço;
- controlar incorretamente algum equipamento.

2. Projeto defeituoso

O sistema pode funcionar exatamente como foi programado, mas possuir uma falha de projeto.

Por exemplo:

Um sistema bancário permite que uma pessoa sem autorização acesse informações de outro usuário.

Nesse caso, o problema não precisa ser simplesmente um "bug".

Pode ser uma **falha de segurança no próprio projeto**.

6. Eficiência

Agora chegamos a um princípio que vai se tornar **central na nossa disciplina**.

Eficiência significa buscar:

obter o resultado desejado utilizando a menor quantidade razoável de recursos.

E aqui tem uma distinção importante:

eficiência não significa simplesmente "fazer mais rápido".

Precisamos considerar os recursos utilizados e o resultado obtido.

Eficiência na Engenharia

Podemos pensar:

Recursos utilizados

Resultado obtido

Quanto menos recursos forem necessários para alcançar determinado resultado, maior tende a ser a eficiência.

Mas o **recurso** depende da situação.

Pode ser:

- energia;
- matéria-prima;
- dinheiro;
- tempo;
- mão de obra;
- espaço;
- capacidade computacional.

7. Eficiência em software

No software, isso fica ainda mais interessante.

Durante o **desenvolvimento**, queremos utilizar menos recursos para produzir o sistema:

- tempo dos programadores;
- infraestrutura;
- dinheiro;
- ferramentas;
- esforço de desenvolvimento.

Durante a **execução**, podemos querer minimizar:

- tempo de processamento;

- uso de CPU;
- memória;
- consumo de energia;
- largura de banda;
- capacidade necessária dos servidores;
- número de operações.

E é aqui que nossa disciplina começa a fazer a ponte para a **análise de complexidade**.

Imagine:

```
Algoritmo A → 1.000 operações  
Algoritmo B → 1.000.000 operações
```

Se ambos resolvem o mesmo problema, existe uma diferença potencialmente enorme no custo de execução.

Portanto:

A forma como escrevemos e estruturamos um algoritmo influencia os recursos necessários para executá-lo.

É justamente por isso que vamos estudar **contagem de operações e tempo de execução**.

8. Manutenibilidade

O quinto princípio é:

Manutenibilidade.

A ideia é projetar o sistema de maneira que futuras intervenções possam ser realizadas com menor dificuldade, tempo e custo.

No mundo físico, podemos pensar em uma máquina que precisa de manutenção.

No software, não temos exatamente o mesmo tipo de desgaste físico.

Um programa não fica "enferrujado" porque foi executado mil vezes.

Porém, software precisa ser alterado constantemente.

Por exemplo:

```
Sistema inicial
  ↓
Correção de bugs
  ↓
Nova funcionalidade
  ↓
Mudança de regra de negócio
  ↓
Integração com outro sistema
  ↓
Nova versão
```

Por isso, a estrutura do código importa muito.

9. Tipos de manutenção

O professor apresentou três categorias.

Preventiva

É planejada antecipadamente para evitar problemas.

No software, isso não funciona exatamente da mesma forma que em máquinas físicas, porque o código não sofre desgaste físico pelo uso.

Preditiva

Baseia-se em monitoramento e previsão para tentar identificar quando uma intervenção será necessária.

Em sistemas de software, podemos ter monitoramento de:

- desempenho;
- uso de recursos;
- comportamento;
- capacidade;
- falhas.

Corretiva

Acontece **depois que um problema é identificado**.

No software:

| **bug → investigação → correção → teste → nova versão**

Essa é uma das formas mais conhecidas de manutenção de software.

10. Manutenção evolutiva

Aqui temos um termo particularmente importante da aula.

A **manutenção evolutiva** acontece quando o software precisa evoluir.

Por exemplo:

Uma empresa possui um sistema de biblioteca.

Versão inicial:

```
Cadastrar livro  
Cadastrar usuário  
Realizar empréstimo
```

Depois surge uma necessidade:

```
Nova funcionalidade:  
Reservar livro
```

O software precisa ser modificado para incorporar essa nova necessidade.

Isso é diferente de simplesmente corrigir um bug.

```
Bug encontrado  
→ manutenção corretiva  
  
Nova necessidade  
→ manutenção evolutiva
```

E o professor destacou que o termo **“manutenção evolutiva”** é **particularmente associado à indústria de software**, diferentemente de outros setores industriais.

11. Programa × Software

Esse é outro conceito que precisamos deixar **muito claro**, porque o professor fez questão de diferenciar os dois.

Na concepção adotada na disciplina:

■ **Programa é o componente material do Software.**

O programa pode estar representado como:

- código-fonte em arquivo de texto;
- código compilado em arquivo binário.

O arquivo pode ser copiado e mantido íntegro.

Mas isso não significa que simplesmente possuir uma cópia do programa seja suficiente para executar todo o software.

Porque um software envolve mais coisas:

```
Software
|
├─ Programa
├─ Dados
├─ Ambiente
├─ Hardware
├─ Sistema operacional
├─ Configurações
└─ Outros componentes necessários
```

Essa distinção vai ser importante porque **a disciplina vai focar no programa/código e, principalmente, nas operações que ele realiza.**

12. A grande ponte: problema → algoritmo → código → operações

E aqui chegamos ao ponto que conecta a introdução inteira com o restante da matéria.

Podemos pensar:

```
PROBLEMA
↓
ALGORITMO
↓
CÓDIGO-FONTE
↓
OPERAÇÕES
↓
TEMPO DE EXECUÇÃO
```

Problema

Temos alguma tarefa que queremos resolver.

Exemplo:

| Encontrar o maior número de uma lista.

Algoritmo

Definimos uma sequência lógica de passos para resolver o problema.

Código-fonte

Um programador transforma esse algoritmo em uma linguagem de programação.

Por exemplo, Python.

Operações

Quando o código é executado, várias operações acontecem:

- atribuições;
- comparações;
- operações aritméticas;
- acessos;
- chamadas etc.

Tempo

Essas operações possuem um custo de execução.

Portanto, podemos estudar:

Quantas operações o programa realiza e quanto tempo essas operações representam?

E é aqui que entra o coração da disciplina

A grande pergunta de Engenharia de Programas começa a ficar clara:

Como estimar o tempo necessário para executar um código?

O professor apresenta uma ideia inicial:

$$T = N_o T_o + N_a T_a + N_c T_c$$

Onde:

- N_o = quantidade de operações;
- T_o = tempo associado às operações;
- N_a = quantidade de atribuições;
- T_a = tempo associado às atribuições;
- N_c = quantidade de comparações;
- T_c = tempo associado às comparações.

Então a lógica geral é:

```
Código-fonte
  ↓
Contar operações
  ↓
Descobrir o custo dessas operações
  ↓
Estimar o tempo
```

E depois a disciplina vai dar um passo além:

E se o tamanho da entrada aumentar?

É aí que começamos a entrar de verdade em **análise de complexidade**.

O fio condutor da matéria

Eu acho que esta é a ideia mais importante para você guardar desde a primeira aula:

Engenharia de Programas procura aplicar princípios de engenharia aos programas, considerando aspectos como viabilidade, racionalidade, segurança, eficiência e manutenibilidade. Dentro dessa perspectiva, a disciplina se concentra especialmente em analisar o custo computacional dos programas, estudando as operações realizadas por algoritmos e como o tempo de execução se comporta conforme o problema cresce.

Então, Ana, não é por acaso que o professor começa falando de viabilidade, segurança e manutenção e depois começa a falar de contagem de operações. Existe uma linha de raciocínio:

```
Engenharia
↓
Princípios de engenharia
↓
Aplicação a software/programas
↓
Eficiência
↓
Algoritmos
↓
Operações
↓
Tempo de execução
↓
Complexidade
```

E essa última parte é justamente onde a disciplina vai ficar mais técnica.